



DSA Notes

Complexity Analysis — Beginner's Cheatsheet



1. What is Time Complexity?

Time Complexity measures how the number of operations grows as the input size (n) increases.

- Focus is on **growth pattern** — not the actual clock time
- Same algorithm can run slower on a slower machine — but the **growth rate stays the same**
- Think of it as: 'If n doubles, how much more work do I do?'



2. Constants (C1, C2, C3...)

Constants represent basic operations that take a fixed amount of time, regardless of input:

- Assignment: `let x = 5`
- Addition: `a + b`
- Comparison: `a > b`
- Array access: `arr[i]`

```
// Each line below = one constant-time operation
let x = 5;           // C1
let y = x + 10;      // C2
if (y > 5) { ... }    // C3

// Total = C1 + C2 + C3 → We just write O(1)
```



Rule: We always ignore constants in the final Big-O answer.

3. Basic Complexities

Code Pattern	Complexity
Constant work	$O(1)$
Single loop	$O(n)$
Nested loops	$O(n^2)$
Halving / Doubling	$O(\log n)$
Mixed ($n \times \log n$)	$O(n \log n)$

Quick memory trick — left = faster, right = slower:

```
 $O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2)$ 
```

4. Loop Patterns

◆ Single Loop

```
for (let i = 1; i <= n; i++) {  
  // runs n times  
}
```

👉 Complexity: $O(n)$

Runs exactly n times. Each increment adds 1 step.

◆ Nested Loop

```
for (let i = 1; i <= n; i++) {  
  for (let j = 1; j <= n; j++) {  
    // runs  $n \times n$  times  
  }  
}
```

👉 Complexity: $O(n^2)$

Outer runs n times. For each outer step, inner runs n times $\rightarrow n \times n$.

♦ Increasing Pattern

```
for (let i = 1; i <= n; i++) {  
  for (let j = 1; j <= i; j++) {  
    // j goes up to i (not n)  
  }  
}  
  
// Total steps: 1 + 2 + 3 + ... + n = n(n+1)/2  $\rightarrow O(n^2)$ 
```

👉 Complexity: $O(n^2)$

Inner loop grows with outer: $1+2+3+\dots+n = n(n+1)/2$. Drop constants $\rightarrow n^2$.

♦ Decreasing Pattern

```
for (let i = 1; i <= n; i++) {  
  for (let j = i; j <= n; j++) {  
    // j starts from i  
  }  
}  
  
// Total steps: n + (n-1) + ... + 1  $\rightarrow O(n^2)$ 
```

👉 Complexity: $O(n^2)$

Mirror of the increasing pattern — still sums to $n(n+1)/2 \rightarrow O(n^2)$.

♦ Doubling Pattern

```
for (let i = 1; i <= n; i *= 2) {  
  // i: 1, 2, 4, 8, 16 ...  
}  
  
// How many times can we double before reaching n?  
// Answer:  $\log_2(n)$  times
```

👉 Complexity: $O(\log n)$

i doubles each step. Reaches n in $\log_2(n)$ steps. Binary search works the same way.

♦ Halving Pattern

```
for (let i = n; i > 1; i /= 2) {  
  // i: n, n/2, n/4, n/8 ...  
}  
  
// How many times can we halve before reaching 1?  
// Answer:  $\log_2(n)$  times
```

👉 Complexity: $O(\log n)$

Same as doubling but in reverse. Still $\log_2(n)$ steps.

♦ Mixed Pattern ($n \log n$)

```
for (let i = 1; i <= n; i *= 2) {    //  $O(\log n)$   
  for (let j = 1; j <= n; j++) {    //  $O(n)$   
    // ...  
  }  
}  
  
// Outer:  $\log n$  steps × Inner:  $n$  steps each  
// →  $n \times \log n = O(n \log n)$ 
```

👉 Complexity: $O(n \log n)$

The outer loop runs $\log n$ times. Each time, inner runs n times → multiply them.

♦ Separate Loops (Addition)

```
for (let i = 1; i <= n; i++) {}    //  $O(n)$   
for (let j = 1; j <= n; j++) {}    //  $O(n)$   
  
// They run one after another → ADD  
//  $O(n) + O(n) = O(2n) \rightarrow O(n)$ 
```

👉 Complexity: $O(n)$

Separate loops add up — but constants are dropped, so $O(2n) = O(n)$.

5. Important Rules

✓ Rule 1: Ignore Constants

- $C1 \rightarrow O(1)$ (a single operation is always constant time)
- $C2*n + C3 \rightarrow O(n)$ (the constant multiplier and added term vanish)

✓ Rule 2: Focus on Highest Power

- $n^2 + n \rightarrow O(n^2)$ (n^2 grows much faster than n , so n becomes irrelevant)
- $n^3 + n^2 + n \rightarrow O(n^3)$ (same logic — keep only the dominant term)

✓ Rule 3: Add vs Multiply

- Separate loops $\rightarrow$ ADD their complexities $\rightarrow O(n) + O(n) = O(n)$
- Nested loops $\rightarrow$ MULTIPLY their complexities $\rightarrow O(n) \times O(n) = O(n^2)$

✓ Rule 4: Identify Loop Behaviour

- $i++$ or $i-- \rightarrow$ linear growth $\rightarrow O(n)$
- $i *= 2 \rightarrow$ doubling $\rightarrow O(\log n)$
- $i /= 2 \rightarrow$ halving $\rightarrow O(\log n)$

6. Mental Model — 3 Questions to Ask

? Question 1: How many times does this block of code run?

? Question 2: Does each step add 1 (linear) or multiply by 2 (logarithmic)?

? Question 3: Are the loops nested (multiply) or separate (add)?

Once you answer these 3 questions, the complexity writes itself.

7. Key Takeaway

Key Takeaway

- We don't measure **exact time** — machines differ
- We measure the **growth of operations** as n increases
- Ignore constants. Drop smaller terms. Keep the dominant one.